Dynamic programming

Part IV Advanced Design and Analysis Techniques

Scope

Dynamic programming merges repeated recursive calls into states. The challenge is proving that a state retains everything needed for future decisions and reconstructing an actual solution from optimal values. Cutting, matrix parenthesization, common subsequences, and search trees develop this method.

Prerequisites

Recursion, optimization, and overlapping subproblems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove optimal substructure for rod cutting and LCS.
- Design state dimensions and evaluation orders.
- Derive interval DP for matrix chains and optimal BSTs.
- Reconstruct choices and reassess DP assumptions in unfamiliar variants.
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rod cutting

Given nonnegative integer length n, p_i is the price of an uncut piece of length i. All pieces have positive integer lengths. Leaving the rod uncut is permitted, but the whole rod must be sold; cuts cost nothing. Maximize revenue.

The first decision cuts off length i and optimally handles n−i. With r_0=0,

`r_n=max_{1<=i<=n}(p_i+r_{n-i})`.

Necessity. In an optimal cutting with value r_n, let the first piece have length i. If the remaining n−i were not optimal, replacing it with a better r_(n−i) cutting would be legal and improve the total, a contradiction.

Sufficiency. Each p_i+r_(n−i) is a legal cutting and cannot exceed the true optimum. The two inequalities establish the recurrence.

Uncached recursion makes $T(0) = 1$ and $T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n$ calls. Memoization or bottom-up evaluation computes each length once, with $\sum_{j=1}^{n} j = \Theta(n^2)$ transitions. T counts calls, not prices or revenue.

Memoization and bottom-up evaluation

Use one-based price array p and evaluate smaller remainders first. Even with negative prices, the whole rod must be sold, so nonempty states start at −∞ rather than silently allowing disposal.

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

At entry to length, all r[0..length−1] are optimal. The inner loop tests every legal first piece, so the recurrence makes r[length] optimal. Reconstruction reduces the remainder by at least1 per step and outputs at most n pieces.

Complete rod table and reconstruction

Let p=[1,5,8,9]. first[n] stores an optimal first-piece length.

n	Candidate revenues	r[n]	first[n]
0	Empty	0	0
1	1	1	1
2	5,1+1=2	5	2
3	8,1+5=6,5+1=6	8	3
4	9,1+8=9,5+5=10,8+1=9	10	2

From n=4, first[4]=2 earns p_2=5 and leaves2. first[2]=2 earns another5 and leaves0. The optimal cutting is2+2, revenue10. Reconstruct after computing the table; do not guess cuts while modifying settled optimal values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete rod table and reconstruction (continued)

A fixed cost c per cut can be subtracted from a candidate only if a remainder remains; one length dimension still suffices. A limit of k pieces requires a piece-count dimension. Add state only when the new restriction affects future feasible decisions, not merely because the specification changed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-chain multiplication

A_i has dimensions $p_{i-1} \times p_i$. If the final multiplication for A_i..A_j splits after k, its cost is

`m[i,k]+m[k+1,j]+p_{i-1}p_k p_j`.

Minimize over i≤k<j, with m[i,i]=0.

Both optimal subchains must be optimal. Replacing the left by cheaper parentheses leaves the final multiplication cost unchanged and improves the whole chain, contradicting optimality; likewise on the right. Every full parenthesization has a unique final split, so enumerating k=i..j−1 is complete.

Fill by increasing length l=j−i+1, since both dependencies are shorter. Store the best k in split[i,j]. There are Θ(n^2) states and O(n) candidates per state, totaling Θ(n^3).

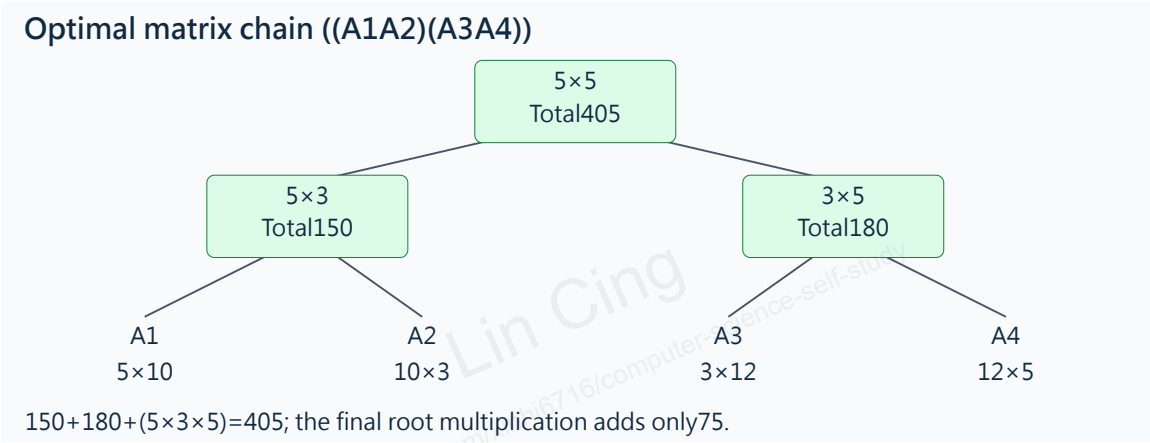
More precisely, the number of candidates is `Σ_(l=2)^n (n−l+1)(l−1)=n(n²−1)/6`. Each uses constantly many reads and arithmetic operations. This proves both bounds; merely multiplying two O bounds would not establish Θ.

Matrix-chain table and parenthesis output

```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

Every dependency is a strictly shorter interval, justifying length order. The reconstruction tree has n leaves and n−1 internal nodes, so emitting parentheses takes Θ(n) node operations, excluding actual matrix-entry multiplication.

Matrix-chain table and parenthesis output (continued)



150+180+(5×3×5)=405; the final root multiplication adds only75.

Complete matrix-chain values and reconstruction

For p=[5,10,3,12,5], the upper-triangular table for A1..A4 is:

Length	Values
1	m11=m22=m33=m44=0
2	m12=150,m23=360,m34=180
3	m13=330,m24=330
4	m14=405

`m24=min(0+180+10*3*5,360+0+10*12*5)=330`, with split[2,4]=2.

`m14=min(0+330+5*10*5,150+180+5*3*5,330+0+5*12*5)=405`, with split[1,4]=2.

That split gives(A1A2) and(A3A4). Their splits1 and3 produce final parenthesization `((A1A2)(A3A4))`.

Elements of dynamic programming

A state must describe the remaining work, not merely rename the original problem. Transitions depend on smaller states. Establish four facts:

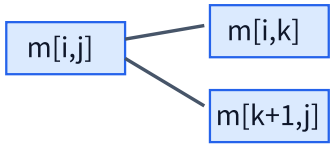
1. Removing the first decision from an optimum leaves an optimum for the corresponding subproblem.
2. Defined substates compose every candidate decision without omitting legal solutions.
3. Base cases are correct and the dependency graph is acyclic.
4. A topological evaluation order completes all prerequisites before a state is filled.

Together these prove recurrence correctness and valid evaluation. Overlap explains why caching saves repeated work; it alone does not prove optimal substructure. Cyclic dependencies require another justified solver rather than one table pass. Reconstruction can store choices or recheck transitions in a full value table; one final scalar optimum usually does not suffice.

Elements of dynamic programming (continued)

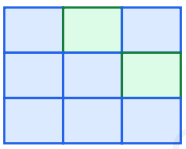
Common structure: a directed acyclic state graph

1 State dependencies



Smaller, acyclic states

2 Fill by length



l=1 → 2 → ⋯

Read shorter intervals

Time = states × choices/state

3 Save choices, backtrack

rod：cut[length]

matrix：split[i,j]

LCS：backpointer

BST：root[i,j]

The state graph determines memoization cost

Rod cutting has n+1 states. Length j depends on0..j−1, totaling n(n+1)/2 edges. Constant work per state/edge gives Θ(V+E)=Θ(n^2).

Top-down memo expands all candidates the first time a state is evaluated and immediately returns its stored value afterward. Keep visited status separate from values, for example None for uncomputed, since a negative revenue can be valid. Bottom-up has no recursion stack; memo may have n stack levels. Both use Θ(n) table space.

The longest-simple-path counterexample does not mean every cycle forbids DP. Endpoint-only states fail to record used vertices. Individually longest simple u→v and v→w paths may both use z, so concatenation repeats z and is not simple. Adding a visited set S permits exponential-state DP; a DAG instead allows longest paths directly in topological order.

Longest common subsequence

Let c[i,j] be the LCS length of prefixes X_i,Y_j. Empty-prefix length is0.

If x_i=y_j, an LCS not ending in that character could append it and become longer. Thus an optimum can end with the common character; removing it gives a candidate for X_(i−1),Y_(j−1). A better prefix subsequence could also append x_i and exceed the original optimum, so the prefix must itself be optimal.

If x_i≠y_j, no common subsequence can depend on both distinct terminal characters as its last character. It omits at least one terminal position, giving

`c[i,j]=max(c[i-1,j],c[i,j-1])`.

The two branches cover every possibility. Each transition costs O(1) and the full table O(mn).

Both LCS inequalities and the full algorithm

When terminal characters match, an LCS not ending with that character uses neither terminal position and can append it, contradicting maximality. Removing the final character bounds its length by c[i−1,j−1]+1. Conversely, any prefix LCS can append the common character legally. Hence c[i,j]=c[i−1,j−1]+1.

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both LCS inequalities and the full algorithm (continued)

The full c table already determines backtracking; a separate direction table is optional. Length alone needs two rows along the shorter string, Θ(min(m,n)) space. This backtracking still requires the full table and cannot run unchanged on only two rows.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking

X=ABCBDAB,Y=BDCABA gives c[7,6]=4. Including empty prefixes:

```
     ε B D C A B A
ε    0 0 0 0 0 0 0
A    0 0 0 0 1 1 1
B    0 1 1 1 1 2 2
C    0 1 1 2 2 2 2
B    0 1 1 2 2 3 3
D    0 1 2 2 2 3 3
A    0 1 2 2 3 3 4
B    0 1 2 2 3 4 4
```

Up decreases i; left decreases j.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking (continued)

Cell	Action	Next	Reverse characters
(7,6)	Up; both candidates4	(6,6)	Empty
(6,6)	A=A, diagonal	(5,5)	A
(5,5)	D≠B; upper3 larger	(4,5)	A
(4,5)	B=B, diagonal	(3,4)	AB
(3,4)	C≠A; left2 larger	(3,3)	AB
(3,3)	C=C, diagonal	(2,2)	ABC
(2,2)	B≠D; left1 larger	(2,1)	ABC
(2,1)	B=B, diagonal	(1,0)	ABCB

Reverse to get BCBA. Each step decreases at least one index, giving O(m+n) backtracking without scanning all mn cells.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking (continued)

LCS values and eight backtracking steps

i↓ / j→	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	4

Green: backtracking path
(7,6) up
(6,6) diagonal, take A
(5,5) up
(4,5) diagonal, take B
(3,4) left
(3,3) diagonal, take C
(2,2) left
(2,1) diagonal, take B

c[7,6]=4; collect A B C B in reverse, giving BCBA.

Optimal binary search trees

Given sorted k1<…<kn, choose a BST shape minimizing expected search cost, including both successful and unsuccessful queries.

Key k_i has probability p_i and unsuccessful interval d_i probability q_i, summing to1. Count visited nodes, so expected cost is

`Σ_i p_i(depth_T(k_i)+1)+Σ_i q_i(depth_T(d_i)+1).`

e[i,j] is optimal cost for k_i..k_j; w[i,j] is all key and dummy probability mass in that interval. An empty interval has one dummy:

`e[i,i-1]=w[i,i-1]=q_{i-1}.`

Choosing root k_r increases every left/right-subtree search depth by1, adding its total weight. Thus

`e[i,j]=min_{i<=r<=j}(e[i,r-1]+e[r+1,j]+w[i,j]).`

Replacing a purportedly optimal subtree with a cheaper one would improve the whole tree. Every root is enumerated, establishing the recurrence.

Optimal-BST table and cost convention

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

This is CLRS's extended-search-tree cost: failed search includes one dummy check. Counting only real-key comparisons gives e[1,n]−Σq_i. This shape-independent constant leaves the optimal tree unchanged. The example's optimum2.10 becomes2.10−0.40=1.70 under real-key comparisons.

Reconstruct build(i,j): if j<i, emit dummy d_(i−1); otherwise r=root[i,j], emit k_r and recursively build(i,r−1),build(r+1,j). Retain empty intervals to deliver the unsuccessful-search leaves correctly.

Complete optimal-BST table and reconstruction

Take p=(0.20,0.30,0.10), q=(0.10,0.05,0.15,0.10).

Interval	w	e	Root
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

For e[1,2], root k2 gives e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45. root[1,3]=2; roots for[1,1] and[3,3] are1,3. Thus k2 has children k1,k3 and dummies attach by BST intervals.

Complete optimal-BST table and reconstruction (continued)

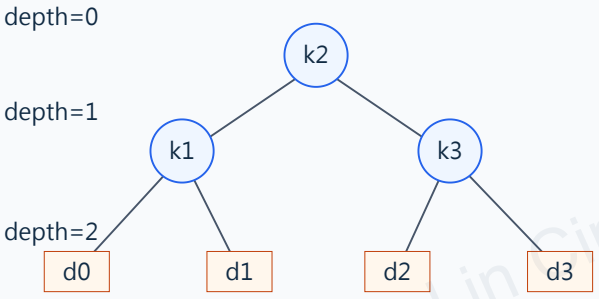
Increasing-length evaluation uses $\Theta(n^2)$ cells each for e,w,root and $\Theta(n^3)$ total candidate roots. This chapter uses that proved version.

Exactly $\Sigma_{l=1}^{n} (n-l+1)l = n(n+1)(n+2)/6$ roots are tested. Each w updates in O(1) from $w[i,j-1]+p_j+q_j$. Rescanning probabilities inside every root candidate would add a factor n and invalidate this bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete optimal-BST table and reconstruction (continued)

Optimal BST and four unsuccessful-search intervals



Expected cost = probability × (depth+1)

k2 : 0.30 × 1 = 0.30

k1,k3 : (0.20+0.10) × 2

d0..d3 : 0.40 × 3 = 1.20

Total0.30+0.60+1.20=2.10

d0: <k1 d1: (k1,k2) d2: (k2,k3) d3: >k3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sequence alignment generalizes LCS

Global alignment allows match, mismatch, and gap. Its last column is one of:

- Paired(x_i,y_j), earning match/mismatch score.
- x_i paired with a gap.
- y_j paired with a gap.

For maximum prefix score F[i,j],

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j),F[i-1,j]+gap,F[i,j-1]+gap).`

This shares LCS's prefix states and backtracking skeleton but needs three score cases. For X=ACGTA,Y=AGCAC, match3,mismatch−2,gap−3, optimum is1 with alignment `ACGTA- / A-GCAC`. Every gap costs its own fixed score. Comparing only equal-length positions misses displacement.

Unbounded knapsack uses capacity states

With repeatable item weights s_i and values v_i, let f[c] be maximum value at **exact** weight c:

`f[0]=0`, `f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i).`

For capacity at most C, maximize over$0 \le c \le C$; exact fill takes only f[C]. Including f[c−1] in the recurrence instead defines value at capacity at most c. Distinguish the meanings.

For A=(3,5),B=(4,7),C=(7,12), capacity13,

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22].`

Optimum22 at weight13 uses A+A+A+B. Highest value/weight greedy chooses B×3 and earns21, disproving general ratio-greedy correctness.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter15, § § 15.1–15.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study